

DRG Cleaning

Carlos Resurreccion

2024-07-01

Load Required Libraries

```
# library(tidyverse)
library(data.table)
library(here)
```

```
## here() starts at C:/Users/resur/Documents/drg-pipeline
```

```
library(tictoc)
```

```
##
## Attaching package: 'tictoc'
```

```
## The following object is masked from 'package:data.table':
##
##      shift
```

```
library(stringr)
library(stringi)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:data.table':
##
##      hour, isoweek, mday, minute, month, quarter, second, wday, week,
##      yday, year
```

```
## The following objects are masked from 'package:base':
##
##      date, intersect, setdiff, union
```

```
library(docstring)
```

```
##
## Attaching package: 'docstring'
```

```
## The following object is masked from 'package:utils':  
##  
##      ?
```

```
library(profvis)  
library(hash)
```

```
## hash-2.2.6.3 provided by Decision Patterns
```

```
##  
## Attaching package: 'hash'
```

```
## The following object is masked from 'package:tictoc':  
##  
##      clear
```

```
## The following object is masked from 'package:data.table':  
##  
##      copy
```

```
# library(foreach)  
# library(doParallel)  
# library(parallel)  
library(future)  
library(future.apply)
```

Set Parameters

Year to Load

```
year_to_load <- "2018"  
version <- "v2"
```

Parameters

```
# Parameters  
sample_size <- 250 * 1e3  
  
drop_cols <- c(paste0("ICDCODE",13:14),"ICCODED15",paste0("ICDCODE",16:17))  
  
icd_cols <- paste0("clin_icd",1:12)  
rvs_cols <- paste0("clin_rvs", 1:20)  
  
to_read <- FALSE  
to_sample <- TRUE  
to_write <- FALSE  
to_group <- TRUE  
to_filter <- FALSE #unused
```

```

to_profvis <- FALSE
to_chunk <- TRUE

na_values <- c('NONE', 'None', '-', '---', 'N/A', 'n/a', 'nan', 'NAN')
na_like_strings <- c("", " ", "-", "none", "None", "NONE", "NA", "n/a",
                     "N/A", "NaN", "", "\t", "\n", "\r", "\f", "\v", "\u00A0",
                     "\u2000", "\u2001", "\u2002", "\u2003", "\u2004", "\u2005",
                     "\u2006", "\u2007", "\u2008", "\u2009", "\u200A", "\u2028",
                     "\u2029", "\u202F", "\u205F", "\u3000")

# File paths
path_to_raw_claims <- "git-ignored-files/raw_claims"
path_to_intermediate <- "git-ignored-files/intermediate_claims"
path_to_cache <- "data-cleaning/cache"
path_to_aux <- "git-ignored-files/aux_files"
path_to_excel <- "git-ignored-files/Excel"
path_to_cleaned_claims <- "git-ignored-files/cleaned_claims"
path_to_grouper_output <- "git-ignored-files/grouper_output"
path_to_chunks <- "git-ignored-files/chunked_samples"

# Here() let's you find files in your project directory

suffix <- paste0(ifelse(to_sample, "_sampled_", "_full_"), version)
sampled_claims <- here(
  path_to_raw_claims,
  paste0("sampled_claims_extract_CLAIMS_", year_to_load,
        suffix, paste0("_", sample_size), ".csv"))
full_claims <- here(
  path_to_raw_claims,
  paste0("claims_extract_CLAIMS_", year_to_load, ".csv"))
intermediate_file <- here(
  path_to_intermediate,
  paste0("intermediate_claims_", year_to_load, "_processed", suffix, ".csv"))
cleaned_claims_file <- here(
  path_to_cleaned_claims,
  paste0("cleaned_claims_extract_CLAIMS_", year_to_load, suffix, ".csv"))
output_txt_file <- here(
  path_to_grouper_output,
  paste0("DRG_Grouped", "_", year_to_load, suffix, ".txt"))
grouper_result_file <- here(
  path_to_grouper_output,
  toupper(paste0("DRG_Grouped", "_", year_to_load, suffix, "Res.TXT")))
total_rows_file <- here(
  path_to_cache,
  paste0("total_rows_", year_to_load, ".rds")
)

# Seed for reproducibility
set.seed(123)

options(future.globals.maxSize= 1024*1024^2)

global_seed <- 123 #for parallelized operations

```

Source Data Formats

```
source(here("data-cleaning", "data-formats.R"))
```

Source Functions

```
source(here("data-cleaning", "functions.R"))
```

Load Mapping Data

```
proc <- fread(here(path_to_excel, "proc.csv"))
proc[, CODE := as.character(CODE)]

rvs_icd9 <- fread(here(path_to_aux, "rvs_icd9cm.csv"),
                  select = c("rvs", "icd9cm"))
rvs_icd9[, rvs := as.character(rvs)]
rvs_icd9[, icd9cm := as.character(icd9cm * 100)]
rvs_icd9 <- merge(rvs_icd9, proc[, .(CODE, DRGUSE)],
                  by.x = "icd9cm", by.y = "CODE", all.x = TRUE)
rvs_icd9[, is_drg := !is.na(DRGUSE) & DRGUSE]
rvs_icd9 <- rvs_icd9[!is.na(rvs) & !is.na(icd9cm), -"DRGUSE"]

acr_rvs <- fread(here(path_to_aux, "acr_rvs.csv"))

# Read in the data.table
tdrg_icd10 <- fread(here(path_to_aux, "i10.csv"))

# Set the key if not already set
setkey(tdrg_icd10, "CODE")

# Subset and assign the result to acc_pdx
acc_pdx <- tdrg_icd10[ACCPDX == "Y", CODE]

# Optional: If you want to ensure uniqueness
# (if CODEs are not unique in tdrg_icd10)
acc_pdx <- unique(acc_pdx)
```

Read Data

Reading Data

```
if (to_read) {
  print("Reading the entire file...")
  dt <- read_entire_file(drop_cols)

  if (to_sample) {
```

```

if (file.exists(sampled_claims)) {
  print("Sampled file exists. Reading the sampled file...")
  dt <- read_sampled_file()
  # Check if the number of rows matches sample_size
  if (nrow(dt) != sample_size) {
    print(paste("Sampled file does not match sample size. Expected:",
      sample_size, "Found:", nrow(dt), "Re-sampling..."))
    dt <- read_entire_file(drop_cols)
    dt <- sample_data(dt)
    if (to_write) {
      print(paste("to_write is TRUE. Writing the new sample data to file:",
        sampled_claims))
      write_data(dt, sampled_claims)
    } else {
      print("to_write is FALSE. Not writing the sample data to file.")
    }
  } else {
    print("Sampled file matches sample size.")
  }
} else {
  print("Sampled file does not exist. Creating new sample...")
  dt <- sample_data(dt)
  if (to_write) {
    print(paste("to_write is TRUE. Writing the new sample data to file:",
      sampled_claims))
    write_data(dt, sampled_claims)
  } else {
    print("to_write is FALSE. Not writing the sample data to file.")
  }
}
} else {
  if (to_sample) {
    if (file.exists(sampled_claims)) {
      print("Sampled file exists. Reading the sampled file...")
      dt <- read_sampled_file()
      # Check if the number of rows matches sample_size
      if (nrow(dt) != sample_size) {
        print(paste("Sampled file does not match sample size. Expected:",
          sample_size, "Found:", nrow(dt), "Re-sampling..."))
        dt <- read_entire_file(drop_cols)
        dt <- sample_data(dt)
        if (to_write) {
          print(paste("to_write is TRUE. Writing the new sample data to file:",
            sampled_claims))
          write_data(dt, sampled_claims)
        } else {
          print("to_write is FALSE. Not writing the sample data to file.")
        }
      } else {
        print("Sampled file matches sample size.")
      }
    } else {

```

```

print("Sampled file does not exist.")
print("Reading entire file and creating new sample...")
dt <- read_entire_file(drop_cols)
dt <- sample_data(dt)
if (to_write) {
  print(paste("to_write is TRUE. Writing the new sample data to file:",
              sampled_claims))
  write_data(dt, sampled_claims)
} else {
  print("to_write is FALSE. Not writing the sample data to file.")
}
}
} else {
  if (!file.exists(intermediate_file)) {
    stop("Cannot proceed: to_read is FALSE and to_sample is FALSE.
         At least one must be TRUE.")
  } else {
    print("Using existing intermediate file.")
  }
}
}
}

```

```

## [1] "Sampled file exists. Reading the sampled file..."
## [1] "Sampled file matches sample size."

```

```

if (file.exists(total_rows_file)) {
  total_rows <- readRDS(total_rows_file)
} else {
  total_rows <- fread(full_claims, select = 1L, header = TRUE)[, .N]
  saveRDS(total_rows, file = total_rows_file)
}

```

Data Processing

Data Cleaning

```

clean_data <- function(dt) {
  original_col_order <- colnames(dt)

  # Add year column
  dt <- add_year_column(dt, year_to_load)
  # Rename columns
  dt <- rename_columns(dt)

  # Process and collapse columns for clin_icd1 to clin_icd12 into clin_icd
  dt <- process_and_collapse_columns(dt, icd_cols, "clin_icd")

  # Process and collapse columns for clin_rvs1 to clin_rvs20 into clin_rvs
  dt <- process_and_collapse_columns(dt, rvs_cols, "clin_rvs")

  # Remove lumped ICD codes from clin_icd

```

```

dt <- remove_lumped_icd_codes(dt, "clin_icd")

# Turn clin_icd and clin_rvs into lists
dt[, clin_icd := split_to_vector(clin_icd)]
dt[, clin_rvs := split_to_vector(clin_rvs)]

# Clean and unlump clin_c1 and clin_c2
dt <- clean_columns_in_dt(dt, c("clin_c1", "clin_c2"))

dt <- remove_lumped_icd_codes(dt, "clin_c1")
dt <- remove_lumped_icd_codes(dt, "clin_c2")

dt[, clin_c1 := split_to_vector(clin_c1)]
dt <- process_icd10_codes(dt, "clin_c1")
dt[, clin_c2 := split_to_vector(clin_c2)]
dt <- process_icd10_codes(dt, "clin_c2")

dt <- append_and_remove_rvs_codes(dt, "clin_c1")
dt <- append_and_remove_rvs_codes(dt, "clin_c2")

dt <- deduplicate_columns(dt, c("clin_c1", "clin_c2", "clin_icd", "clin_rvs"))

# Replace empty strings in character and factor columns with NA
dt <- replace_empty_with_na(dt)

# Modify factor cols
dt <- process_patient_type(dt)
dt <- process_memcat_parent_desc(dt)
dt <- process_memcat_child_desc(dt)
dt <- process_disposition(dt)
return(dt)
}

```

```

if (!to_chunk){
  if (to_profvis) {
    profvis({
      tic("Total execution time:")
      # plan(multisession, workers = availableCores() - 1)
      dt <- clean_data(dt)
    })
  } else {
    tic("Total execution time:")
    # plan(multisession, workers = availableCores() - 1)
    dt <- clean_data(dt)
  }
}

```

Not chunking

```

if (to_chunk){
  if (to_profvis) {
    profvis({
      tic("Total execution time:")
      num_cores <- max(1, availableCores() - 1)

      chunk_size <- ceiling(nrow(dt) / num_cores)
      chunks <- split(dt, rep(1:num_cores, each = chunk_size,
                             length.out = nrow(dt)))

      # Plan for parallel processing
      plan(multisession, workers = num_cores)

      # Process each chunk in parallel
      processed_chunks <- future_lapply(chunks, process_chunk,
                                         future.seed = global_seed)

      # Combine processed chunks
      dt <- rbindlist(processed_chunks)

      dt <- replace_empty_with_na(dt)
    })
  } else {
    tic("Total execution time:")
    num_cores <- max(1, availableCores() - 1)

    chunk_size <- ceiling(nrow(dt) / num_cores)
    chunks <- split(dt, rep(1:num_cores, each = chunk_size,
                           length.out = nrow(dt)))

    # Plan for parallel processing
    plan(multisession, workers = num_cores)

    # Process each chunk in parallel
    processed_chunks <- future_lapply(chunks, process_chunk,
                                       future.seed = global_seed)

    # Combine processed chunks
    dt <- rbindlist(processed_chunks)

    dt <- replace_empty_with_na(dt)
  }
}

```

Chunking

If not chunking, map codes and find pdx

```

#Map RVS codes
if (!to_chunk){

```



```

if (to_profvis) {
  profvis({
    dt <- process_rvs_code_mapping(dt, rvs_icd9)
    dt <- process_icd10_mapping(dt)
    # Replace empty strings in character and factor columns with NA
    dt <- replace_empty_with_na(dt)
  })
} else {
  dt <- process_rvs_code_mapping(dt, rvs_icd9)
  dt <- process_icd10_mapping(dt)
  # Replace empty strings in character and factor columns with NA
  dt <- replace_empty_with_na(dt)
}
}

```

Map Codes

```

#Map RVS codes
if (!to_chunk) {
  if (to_profvis) {
    profvis({
      dt <- apply_find_pdx(dt, acc_pdx)
    })
  } else {
    dt <- apply_find_pdx(dt, acc_pdx)
  }
}

```

```

if (to_write) {
  fwrite(dt, here("data-cleaning", "output.csv"))
}

```

Find PDx

Export for Batch Grouper

```

if (to_group) {
  if (to_profvis) {
    profvis({
      export_for_batch_grouper(dt, year_to_load, output_txt_file)
      for_batch_grouping <- fread(output_txt_file,
                                   sep = "|", na.strings = "--")
      batch_grouping_result <- fread(grouper_result_file,
                                   sep = "|", na.strings = "--")
    })
  } else {

```

```

    export_for_batch_grouper(dt, year_to_load, output_txt_file)
    for_batch_grouping <- fread(output_txt_file,
                                sep = "|", na.strings = "--")
    batch_grouping_result <- fread(grouper_result_file,
                                sep = "|", na.strings = "--")
  }
}

```

Stop Timer & Calculate Speed

```

# Stop the timer and capture total time
toc_data <- toc(log = TRUE)

```

```

## Total execution time:: 38.15 sec elapsed

```

```

total_time <- toc_data$toc - toc_data$tic

```

```

# Calculate time spent per cell and per row
total_rows_dt <- nrow(dt)
total_cells <- nrow(dt) * ncol(dt)

```

```

time_per_cell <- total_time / total_cells
time_per_row <- total_time / total_rows_dt
time_estimate_total_rows <- time_per_row * total_rows

```

```

# Format the row numbers
formatted_total_rows_dt <- format_large_numbers(total_rows_dt)
formatted_total_rows <- format_large_numbers(total_rows)

```

```

# Print the results with aligned decimal points and formatted row numbers
cat(sprintf("Time spent (total) for %2s rows: %1.2f sec (actual)\n",
            formatted_total_rows_dt, total_time))

```

```

## Time spent (total) for 250.0k rows: 38.15 sec (actual)

```

```

cat(sprintf("Time spent (t/row) for %2s rows: %1.2f msec (actual)\n",
            formatted_total_rows_dt, time_per_row * 1000))

```

```

## Time spent (t/row) for 250.0k rows: 0.15 msec (actual)

```

```

cat(sprintf("Time spent (total) for %2s rows: %2.2f min (estimate)\n",
            formatted_total_rows, time_estimate_total_rows / 60))

```

```

## Time spent (total) for 11.8m rows: 29.95 min (estimate)

```